9

BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Piggybacking on volatile: Güvenli Durum Yayınlama (Safe Publication)

Piggybacking on volatile: Safe State Publication & Ordering Guarantees

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              PIGGYBACKING (SIRTA BİNME) HAPPENS-BEFORE KURALI │
└──────────────────────────────────────────────────────────────┘

NON-VOLATILE DEĞİŞKENLER:              VOLATILE DEĞİŞKEN (KORUYUCU MELEK):
int a = 10;
String name = "MAANG";
List<String> items = new ArrayList();
items.add("Data");
                                       volatile int trigger = 1; // ◄── YAZMA!
──────────────────────────────────────────────────────────────────
BÜTÜN BU NORMAL DEĞİŞKENLER, TRIGGER'IN ARKASINA TAKILARAK RAM'E BASILIR!

OKUYUCU THREAD:
if (trigger == 1) { // ◄── VOLATILE OKUMA!
    // a, name ve items listesi KESİNLİKLE tam ve güncel olarak görünür!
}
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Havaalanında uçağa binerken pasaportu olmayan 3 çocuk düşünün (Normal değişkenler). Kapıdaki polis tek tek çocukları içeri almaz. Ama yanlarında diplomatik pasaporta sahip bir büyükelçi (volatile değişken) vardır. Büyükelçi kapıdan geçerken 'Bu çocuklar benimle' dediği anda, bütün çocuklar büyükelçinin sırtına binerek (Piggybacking) içeriye güvenle sızarlar! Tek bir volatile değişken, kendisinden önce yazılan onlarca normal değişkeni de beraberinde görünür kılar!

🇬🇧 ENGLISH (ELI5)

Imagine three passengers lacking boarding passes (regular non-volatile fields). Airport security refuses them entry. However, they walk alongside an accredited diplomat holding an all-access pass (a single `volatile` variable). When the diplomat clears the gate, the Happens-Before protocol dictates that everyone preceding the diplomat in line is admitted simultaneously (Piggybacking). A single volatile write flushes all preceding non-volatile state mutations into global visibility!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Piggybacking Mekanizması ve JMM Transitivity Kuralı: 1) Happens-Before Transitivity: Eğer `A -> B` (Program Order) ve `B -> C` (Volatile kuralı) ise, o zaman `A -> C`'dir. Burada A normal değişken yazmaları, B volatile yazma, C ise başka bir thread'in o volatile değişkeni okumasıdır. 2) Java Standart Kütüphanesindeki Yeri: `java.util.concurrent` paketindeki birçok sınıf (ör. `AbstractQueuedSynchronizer` - AQS ve `ConcurrentHashMap`) bu hileyi yoğun şekilde kullanır. Tüm alanları `volatile` yapmak yerine, tek bir `volatile int state` değişkenine yazarak sınıftaki diğer tüm düz alanları görünür kılarlar. Bu sayede gereksiz bellek bariyeri maliyetlerinden kaçınarak inanılmaz bir throughput elde edilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Piggybacking Formalism & JDK Engineering: 1) Transitively Inherited Visibility: JSR-133 guarantees that if Action A (non-volatile writes) happens-before Action B (volatile write), and Action B happens-before Action C (volatile read by Thread 2), then Action A happens-before Action C. 2) High-Performance JDK Exploitation: `AbstractQueuedSynchronizer` (AQS) powers ReentrantLock, Semaphore, and CountDownLatch. AQS does not mark internal queue node pointers as volatile; instead, it relies on piggybacking off mutations to `volatile int state` to safely publish entire linked-node updates, evading catastrophic memory fence penalties.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Piggybacking ile Düz Değişkenleri Güvenle Yayınlama
import java.util.HashMap;
import java.util.Map;

public class PiggybackingPublisher {
    // Bu map normal bir HashMap'tir (Thread-safe DEĞİLDİR!)
    private Map<String, String> internalConfig = new HashMap<>();

    // Sihirli koruyucu: Tek bir volatile tetikleyici
    private volatile boolean published = false;

    public void prepareAndPublish() {
        // 1. Normal, senkronize olmayan map manipülasyonu
        internalConfig.put("db.user", "admin");
        internalConfig.put("db.password", "secret");
        internalConfig.put("cache.ttl", "3600");

        // 2. Piggybacking tetikleyici: Volatile yazma!
        // Kendisinden önceki BÜTÜN map verilerini bellek bariyeriyle RAM'e döker.
        published = true;
    }

    public void readConfig() {
        if (published) { // Volatile okuma
            // internalConfig KESİNLİKLE eksiksiz ve güvenle okunur!
            IO.println("Kullanıcı: " + internalConfig.get("db.user"));
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Kritik Kodlama Tuzağı: Piggybacking yaparken kod sıralamasına çok dikkat edilmelidir! Eğer `published = true;` satırını map'e veri eklemeden ÖNCE yazarsanız, reordering yüzünden okuyucu thread boş veya yarım doldurulmuş map ile karşılaşır!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Dangerous Trap: The order of statements is lethal in piggybacking! If you assign `published = true` BEFORE mutating `internalConfig`, the happens-before chain is severed. Observers will read partially constructed, corrupted data structures.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Piggybacking, normal değişkenleri volatile bir yazmanın arkasına takarak görünür kılmaktır.
- JMM'nin Geçişlilik (Transitivity) kuralı sayesinde A -> B ve B -> C ise A -> C olur.
- JDK'nın AQS altyapısı yüksek performans için piggybacking prensibini kullanır.

🎯 Key Takeaways for the Interview

- Piggybacking piggybacks non-volatile updates onto the back of a volatile write.
- Happens-before transitivity legally guarantees that preceding non-volatile state is visible.
- JDK concurrency engines (AQS) leverage piggybacking to minimize memory fence overhead.